```
from mutagen.mp3 import MP3
# If title is missing set it to the file name without the
extension.
def get_file_tags(f):
    mp3 = MP3(f)
    try:
        title=mp3['TIT2'][0]
    except:
        title=f.split('/')[-1][:-4]
    try:
        artist=mp3['TPE1'][0]
    except:
        artist="Unknown"
    try:
        rating=mp3['POPM:'].rating
    except:
        rating="0"
    return (title, artist, rating)
```

The Mutagen module gives you the tags similar to the key and value pairs of a dictionary. The key is as per ID3 tag names. If the tag is missing, it raises a key error exception.

The corresponding function when using the eyeD3 module is as follows:

```
import eyed3
def get_file_tags(f):
    mp3 = eyed3.load(f)
    try:
        title=mp3.tag.title
        artist=mp3.tag.artist
    except:
        print f
        title=f.split('/')[-1][:-4]
        artist='Unknown'
    rating = 0
    for popm in mp3.tag.popularities:
        if popm.rating:
            rating = popm.rating
            break
    return (title, artist, rating)
```

This module gives us the values as a tag record with recognisable field names, e.g., *tag.title*. The popularities tag is a list of three values – email, rating and play-count. Multiple entries can be present, one for each email. This can be useful if many people are using a common music library.

The eyeD3 and the Mutagen modules behave differently if a tag like *title* or *artist* is missing. EyeD3 returns a value of *None* while Mutagen raises an exception.

Since the patents on MP3 have expired, there is no strong motive to avoid MP3 files and use OGG instead. Hence, you may decide to stick to eyeD3 module as that seems to be simpler to use.

Updating the tags

After correcting the artist and title tags and entering your ratings, you are ready to modify the file tags. The rating is a number between 0 and 255.

The following is the code for the *update_tags* option, using the eyeD3 module. The modified CSV file is converted into a dictionary with the file name as the key. You assign the revised values to the title and artist tags. The popularities tag is set by providing the email, rating and play-count values.

```
def set_file_tags(f,tags):
    mp3 = eyed3.load(f)
    mp3.tag.title=unicode(tags[0])
    mp3.tag.artist=unicode(tags[1])
    mp3.tag.popularities.set('',int(tags[2]),0)
    mp3.tag.save()
def update_tags(path):
    """ Convert the csv file into a dictionary
        key is the file name, the 1st column
        value is a list of values of the remaining columns"""
    dtags={}
    for line in open('mp3.csv'):
        fields=line[:-1].split(';')
        dtags[fields[0]]=fields[1:]
    for f in get_mp3_file(path):
        set_file_tags(f,dtags[f])
```

Renaming the music files

The final option to rename the files as per your personal preferences using the tag values is as follows:

```
def rename_file(f):
    """ Standardise the file names as artist-title.mp3"""
    mp3 = eyed3.load(f)
    root,fold = split(f)
    try:
        fnew = "%s-%s.mp3"%(mp3.tag.artist, mp3.tag.title)
        os.rename(f,join(root,fnew))
    except:
        print f
def rename(path):
    for fn in get_mp3_file(path):
        rename_file(fn)
```

All this can of course be done without coding, but by using a music application and the file manager. The time required will be less, unless your music library is really large. However, automation by coding is definitely more fun and saving time is just an excuse—*https://xkcd.com/1205/*. END

By: Dr Anil Seth

The author has earned the right to do what interests him. You can find him online at *http://sethanil.com* and *http://sethanil.blogspot.com*, or you can reach him via email at *anil@sethanil.com*.